Class 23: Process Models and Agile Foundations

Data-driven Systems Engineering

Process models are decision tools

Compare

Compare plan-driven, incremental, and hybrid approaches.

Hybrid for ML

Understand why ML projects often need hybrid processes.

Match the context

Match process choice to uncertainty, regulation, and team maturity.

Trace the effects

Explain how process decisions affect documentation, testing, and release risk.

Bridge from Class 22. Requirements will change; the process decides how a team absorbs that change without losing control.

RUNNING CASES

fraud scoring API | pasteurization monitoring | hospital ML tool (contrast)

NEW **Process model:** a description of how software work is organized: activities, products, roles, and conditions.
Hybrid process: mixes plan-driven control with agile iteration.

Roadmap: three questions

1 Process families

What does a process control?

- Plan-driven, incremental, agile
- The same four activities, organized differently
- The V-model

2 Agile foundations

What did agile change, and what did it keep?

- History and the manifesto
- Principles adapted to ML teams
- Definition of done and failure modes

3 Choosing for ML

Which process fits an ML system, and why?

- Hybrid delivery and CRISP-DM
- Gates between exploration and production
- How to choose; a project checklist

Key idea. A process is chosen, not inherited: each part gives you one tool for defending that choice in your project.

NEW **Process family:** a group of processes that organize the same activities in a similar way.

1 Process families



Previously: class 22 made requirements testable and traceable; the process decides how a team absorbs changes to them.

IN THIS PART

- What a process model controls
- The same four activities, organized differently
- Plan-driven, incremental, agile: when does each fit?
- The V-model: every stage has its check

A process model makes coordination explicit

A process model answers four questions: who decides, who reviews, what evidence is required, and how work moves between steps.

Activities

Data modeling, UI design, service integration.

Products

Documents, code, models, dashboards, tests.

Roles

Analyst, developer, data scientist, reviewer, operator.

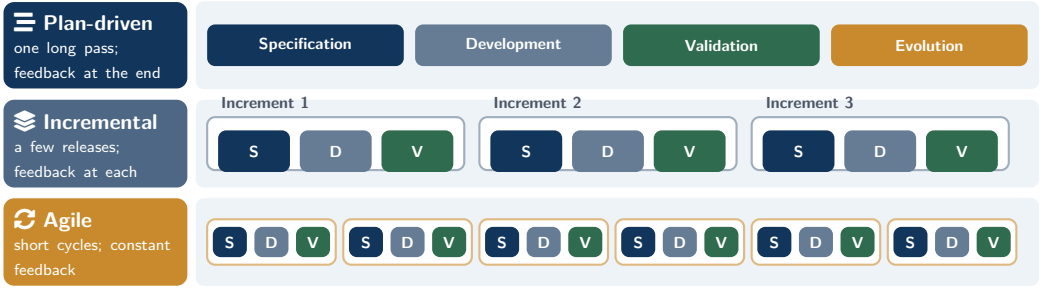
Conditions

What must be true before an activity starts, and after it ends.

Key idea. Process is not bureaucracy by definition: a good process reduces friction by clarifying how work moves from idea to dependable outcome. In ML systems it also separates exploratory work from production commitments.

NEW **Handoff:** passing work from one person or step to the next. **Pre- and post-condition:** what must be true before an activity starts and after it ends.

The same four activities, organized differently



Key idea. The activities are the same; what changes is the length of the loop and when feedback arrives.

NEW **Plan-driven:** activities planned in advance, with progress measured against the plan. **Increment:** a usable piece of the system delivered early. **Iteration:** one short cycle of specifying, building, and checking.

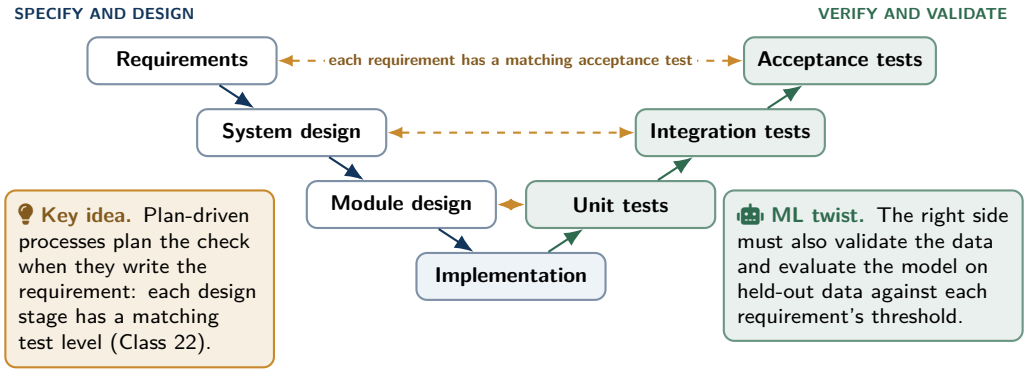
Plan-driven, incremental, agile: when does each fit?

	Plan-driven	Incremental	Agile
IDEA	Plan first; measure progress against the plan.	Deliver usable increments; learn early.	Customer feedback keeps reshaping priorities.
STRENGTH	Control, documentation, auditability.	Small releases: cheaper to be wrong, easier to roll	Fast feedback and iteration.
MAIN RISK	Adapts poorly to late discoveries.	Fails without a strong definition of done.	Drifts without disciplined scope control.
FITS WHEN	Stable scope; regulated or safety-critical.	Frequent releases; CI/CD in place.	Uncertain requirements or data.

Hybrid. Exploration plus governance: agile where uncertainty is high, plan-driven control where risk is high, with explicit handoff rules.

NEW **Hybrid process:** mixes plan-driven control with agile iteration. **Auditability:** how easily an outside reviewer can check what was done and why.

The V-model: every stage has its check



NEW **V-model:** a plan-driven process that pairs each design stage with a test level. **Test level:** the scope of a check: one unit, several parts together, or the whole system. **Validation:** checking that we built the right thing.

2 Agile foundations



Previously: plan-driven, incremental, and agile processes arrange the same four activities in loops of different length.

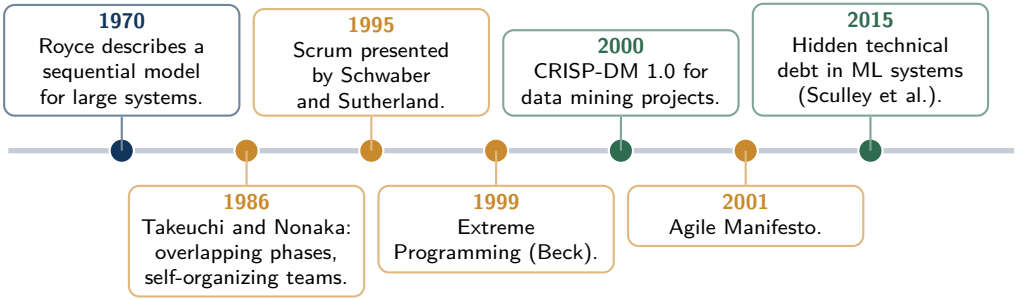
IN THIS PART

- Why agile emerged
- The manifesto: four values
- Agile principles adapted to ML teams
- Definition of done, and the failure modes of agile

Why agile emerged

Agile methods grew partly from dissatisfaction with heavyweight methods in the 1980s and 1990s: teams wanted faster delivery, lower overhead, and a better response to changing requirements. They never meant "no design" or "no documentation": both should earn their cost.

Not to scale. Blue: plan-driven Gold: agile Green: data and ML



NEW **Scrum:** an agile framework with fixed-length cycles and defined roles. **Extreme Programming:** an agile method built on engineering practices such as testing and pair programming.

The Agile Manifesto: four values



Individuals and interactions
over processes and tools



Working software
over comprehensive documentation



Customer collaboration
over contract negotiation



Responding to change
over following a plan

Key idea. The right side still matters. In ML projects, documentation, governance, and change control return quickly once deployment begins.

The manifesto also lists twelve principles; the next slide uses Sommerville's five, adapted to ML teams.

NEW **Agile Manifesto:** a 2001 statement of four values for software development. **Change control:** formal approval of what may change, and when.

Agile principles adapted to ML teams

PRINCIPLE	WHAT IT MEANS	IN ML TEAMS
Customer involvement	Customers take part throughout: they set priorities and evaluate each increment.	**Stakeholders define value and review increments.**
Incremental delivery	Build in increments; the customer specifies the requirements of each.	**Baselines, APIs, dashboards, and monitoring in stages.**
People over process	Recognize the team's skills and let it choose its own ways of working.	**Expertise matters while data is uncertain.**
Embrace change	Expect requirements to change, and design so the system can absorb it.	**Feedback from errors, labels, and operations is expected.**
Maintain simplicity	Keep both the software and the process simple; remove complexity.	**Avoid needless features, pipelines, and tools.**

Five principles after Sommerville, *Software Engineering*; the right column adapts them to ML.

NEW **Baseline:** a first, simple model used as the reference for later versions.

Definition of done: when is an increment really finished?

EXAMPLE DONE CHECKLIST (ML)

- [x] Code reviewed; tests pass.
- [x] Data checks pass (schema, ranges, gaps).
- [x] Model meets its threshold on held-out data.
- [x] Model and data versions logged.
- [x] Docs and fallback behavior updated.

Definition of done. One shared checklist that every increment must pass before it counts as finished.

Acceptance criteria. Conditions specific to one requirement or story: the behavior that this item must show.

Key idea. An item is done when its acceptance criteria pass and the definition of done is met. Without a shared definition of done, iteration turns into churn.

NEW **Definition of done:** a shared checklist an increment must meet to count as finished. **Acceptance criteria:** conditions specific to one requirement (Class 22).

Drawbacks and failure modes of agile

FAILURE MODE	WHAT HAPPENS	TYPICAL GUARD
⧗ Disengaged customers	Customers may disengage from the constant feedback cycle.	**A short, fixed review rhythm with visible increments.**
⤮ Conflicting priorities	Multiple stakeholders may prioritize conflicting changes.	**One owner for a single, ordered backlog.**
🚀 Speed without discipline	Teams can confuse speed with lack of discipline.	**Tests and quality gates inside the definition of done.**
↻ Churn	Without clear definitions of done, iteration turns into churn.	**A shared, written definition of done.**

Guards are common practices, not guarantees.

NEW **Backlog:** an ordered list of the work still to do. **Churn:** rework that does not move the system forward.

3 Choosing for ML

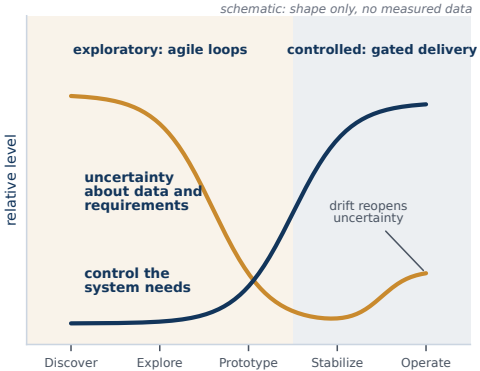


Previously: agile foundations: increments, a shared definition of done, and the failure modes to guard against.

IN THIS PART

- Why ML work often needs hybrid delivery
- CRISP-DM: a process model made for data work
- A hybrid lifecycle with gates
- Where do our projects sit, and how to choose a process
- A project checklist

Why ML work often needs hybrid delivery



Early phases are exploratory: data quality, labels, and metrics are uncertain.

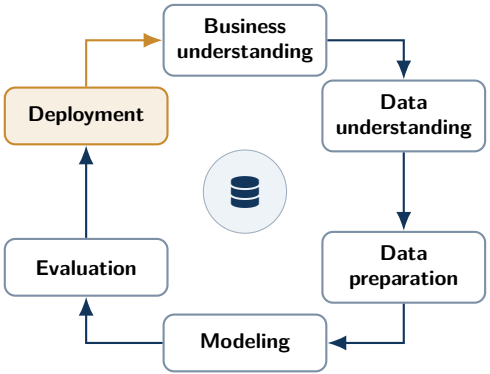
Later phases need discipline: deployment, interfaces, testing, and compliance.

Regulated contexts cannot rely on verbal agreements or informal notebooks.

Example. An industrial monitoring system may use agile prototyping for the predictor, then a more controlled process for deployment in the plant network.

NEW **Exploratory work:** work whose goal is to learn what is possible, not to deliver. **Governance:** rules, reviews, and approvals that control change.

CRISP-DM: a process model made for data work



Iteration is built in: the phase sequence is not strict, and moving back and forth is expected.

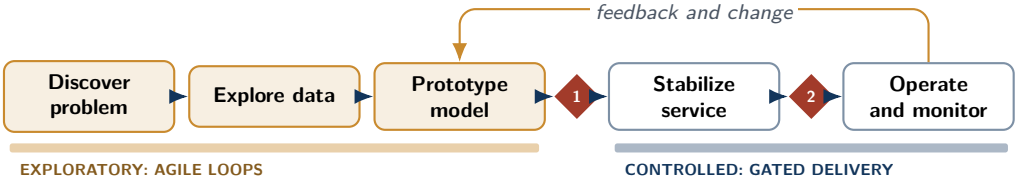
Evaluation checks the results against the business objective, not only against a metric.

The last phase **plans monitoring and maintenance**; running models in production is the subject of MLOps (Class 26).

CRISP-DM 1.0 dates from 2000 and was written for data mining projects.

NEW **CRISP-DM:** Cross-Industry Standard Process for Data Mining: six phases for data projects. **Business objective:** the outcome the organization wants, stated before any modeling.

A hybrid lifecycle with gates



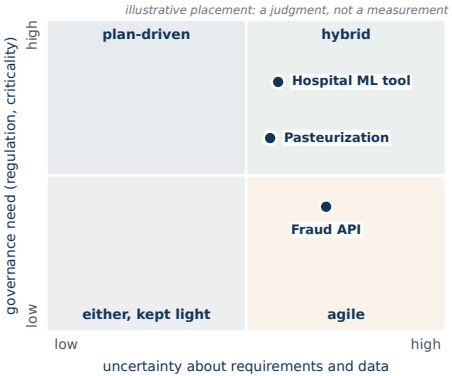
Gate 1: prototype to stabilize. Example exit criteria: stakeholders accept a metric and a data source, and the requirements are testable.

Gate 2: stabilize to operate. Example exit criteria: tests pass, the interface contract is stable, validation evidence is logged, and a fallback is defined.

Key idea. Match control to uncertainty: loops where the team is still learning, gates where a mistake becomes expensive.

NEW **Gate:** a checkpoint where evidence decides whether work may move to the next stage. **Exit criteria:** the conditions that must hold to pass a gate. **Handoff rule:** who passes what to whom, and with what evidence.

Where do our projects sit?



Fraud API project

Agile for feature and threshold iteration; structured release gates; monitoring after deployment.

Pasteurization case

Exploratory modeling can be agile; deployment gets stronger validation and change control.

Hospital ML support tool

Stronger requirement control; formal validation and logging; slower but safer release cadence.

NEW **Criticality:** how much harm a failure can cause. **Release cadence:** how often new versions go live.

How to choose a process model

QUESTION	PROCESS IMPLICATION
Are requirements stable?	More stability supports plan-driven structure.
Is data quality uncertain?	More uncertainty favors iterative exploration.
Is the system regulated?	More regulation requires stronger documentation and approval.
Are releases frequent?	Frequent change favors incremental delivery and automation.
Will the model evolve in production?	Evolution demands monitoring and change governance.

Key idea. These questions are an ML version of Boehm and Turner's five factors: size, criticality, dynamism, personnel, and culture. Rate your project on each before choosing.

NEW **Boehm and Turner:** they rate a project on five factors to place it between agile and plan-driven.

Project checklist: what to defend in your presentation

1. Why did you choose this process?
2. Where may the requirements still change?
3. Which artifacts keep the team aligned?
4. When do you move from experimentation to controlled delivery?

Key idea. Be ready to defend each answer with an artifact from your project, not only with an argument.

NEW **Artifact:** anything the team produces that keeps others aligned: a document, a test, a dashboard.

From this class to the next

1 SE and ML | 2 Requirements | **3 Process** | 4 Design | 5 Quality | 6 Evolution

THIS CLASS AND CLASS 25

What stays with us

- No process is best everywhere; fit depends on uncertainty and governance.
- ML needs iterative discovery without losing engineering control.

Use it in your project

- Justify whether your work needs more loops, more structure, or a hybrid.
- Define your gates and your definition of done.

Next: Class 25

Agile Toolkit.

Backlogs, stories, boards, ceremonies, and feedback loops. Class 24 is a guest session (YesAlps).

Course thread. After choosing a development posture, we turn it into everyday coordination practices a team can actually run.

References

Books and guides

Sommerville, I. *Software Engineering*, Pearson.

Boehm, B. and Turner, R. (2003). *Balancing Agility and Discipline: A Guide for the Perplexed*.

Schwaber, K. and Sutherland, J. The Scrum Guide. Agile Manifesto.

History

Royce, W. W. (1970). Managing the Development of Large Software Systems. *Proc. IEEE WESCON*.

Takeuchi, H. and Nonaka, I. (1986). The New New Product Development Game. *Harvard Business Review*.

Fowler, M. The New Methodology.

Data and ML process

Chapman, P. et al. (2000). *CRISP-DM 1.0: Step-by-step data mining guide*.

Sculley, D. et al. (2015). Hidden Technical Debt in Machine Learning Systems. NeurIPS.

Paleyes, A. et al. (2022). Challenges in Deploying Machine Learning: a Survey of Case Studies. *ACM Computing Surveys*.